用户模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 用户表(user)

User类：对应用户表，包含用户基本信息属性
【springboot/src/main/java/org/example/springboot/entity/User.java】

```
@Data
@TableName("user")
@Schema(description = "用户实体类")
public class User {
    @TableId(type = IdType.AUTO)
    @Schema(description = "用户ID")
    private Long id;

    @Schema(description = "用户名")
    @NotBlank(message = "用户名不能为空")
    @Size(min = 3, max = 50, message = "用户名长度必须在3到50个字符之间")
    private String username;

    @Schema(description = "密码")
    @NotBlank(message = "密码不能为空")
    @Size(min = 6, max = 100, message = "密码长度必须在6到100个字符之间")
    private String password;

    // 其他属性...
}
```

1.1.2 角色表(sys_role)

Role类：对应角色表，定义用户角色及权限
【springboot/src/main/java/org/example/springboot/entity/Role.java】

```
@Data
@TableName("sys_role")
@Schema(description = "角色实体")
public class Role {
    @TableId(type = IdType.AUTO)
    @Schema(description = "角色ID")
    private Integer id;
```

半夜撕代码版权所有

```java
    @NotBlank(message = "角色编码不能为空")
    @Size(min = 2, max = 50, message = "角色编码长度必须在2到50个字符之间")
    @Schema(description = "角色编码")
    private String code;

    @NotBlank(message = "角色名称不能为空")
    @Size(min = 2, max = 50, message = "角色名称长度必须在2到50个字符之间")
    @Schema(description = "角色名称")
    private String name;

    // 其他属性...
}
```

1.1.3 菜单表(sys_menu)

Menu类：对应菜单表，定义系统菜单结构
【springboot/src/main/java/org/example/springboot/entity/Menu.java】

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
@Schema(description ="菜单类实体")
@TableName("sys_menu")
public class Menu {
    @Schema(description ="菜单id")
    @TableId(type = IdType.AUTO)
    private Integer id;

    @Schema(description ="菜单名")
    private String name;

    @Schema(description ="菜单路径")
    private String path;

    // 其他属性...
}
```

1.2 核心属性说明

1. 用户表核心属性
    - id：用户唯一标识，自增主键
    - username：用户名，唯一，长度3-50
    - password：密码，使用BCrypt加密存储，长度6-100
    - email：邮箱，唯一，必填
    - phone：手机号，格式为中国大陆手机号

半夜撕代码版权所有

- roleCode：角色编码，关联角色表
- status：状态(0:禁用,1:正常)
- token：非数据库字段，JWT令牌
- menuList：非数据库字段，用户菜单权限列表

2. **角色表核心属性**
   - code：角色编码，唯一标识符
   - name：角色名称
   - menuList：非数据库字段，角色关联的菜单列表
3. **菜单表核心属性**
   - path：菜单路径
   - component：组件路径
   - pid：父级菜单ID
   - hidden：是否隐藏
   - children：非数据库字段，子菜单列表

二、各功能详细讲解

2.1 用户注册功能

2.1.1 功能概述

用户注册功能允许新用户通过填写用户名、密码、邮箱等信息创建账户。系统会验证用户输入的合法性，确保用户名和邮箱的唯一性，并对密码进行加密存储，增强系统安全性。

2.1.2 详细讲解功能实现流程

用户注册流程始于前端Register.vue组件，用户填写注册表单后点击提交按钮，触发handleRegister方法。该方法首先通过Element UI的表单验证机制检查输入的合法性，包括用户名长度、密码强度、邮箱格式等。验证通过后，将表单数据通过request.post("/user/add", registerData)发送到后端。

后端UserController接收请求，调用userService.createUser(user)方法处理注册逻辑。在UserService中，系统首先检查用户名和邮箱是否已被使用，然后使用BCryptPasswordEncoder对密码进行加密，最后将用户信息保存到数据库。注册成功后，前端收到成功响应并跳转到登录页面。

2.1.3 关键代码讲解

1. **前端注册表单验证**
   【vue3/src/views/auth/Register.vue】

半夜撕代码版权所有

```
const rules = {
  username: [
    { required: true, message: t('auth.validation.usernameRequired'),
trigger: 'blur' },
    { min: 3, max: 50, message: t('auth.validation.usernameLength'),
trigger: 'blur' }
  ],
  password: [
    { required: true, message: t('auth.validation.passwordRequired'),
trigger: 'blur' },
    { min: 6, max: 100, message: t('auth.validation.passwordLength'),
trigger: 'blur' }
  ],
  // 其他验证规则...
}
```

这段代码定义了注册表单的验证规则，确保用户名和密码符合长度要求，邮箱格式正确，密码确认与原密码一致，手机号格式正确。

2. 后端用户创建逻辑
【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public void createUser(User user) {
    // 检查用户名是否存在
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
                .eq(User::getUsername, user.getUsername())
        ) != null) {
        throw new ServiceException("用户名已存在");
    }

    // 检查邮箱是否被使用
    if (userMapper.selectOne(
            new LambdaQueryWrapper<User>()
                .eq(User::getEmail, user.getEmail())
        ) != null) {
        throw new ServiceException("邮箱已被使用");
    }

    // 密码加密
    user.setPassword(bCryptPasswordEncoder.encode(user.getPassword()));

    if (userMapper.insert(user) <= 0) {
        throw new ServiceException("用户创建失败");
    }
}
```

半夜撕代码版权所有

这段代码实现了用户创建的核心逻辑：检查用户名和邮箱唯一性，对密码进行BCrypt加密，然后将用户信息插入数据库。如果任何步骤失败，都会抛出ServiceException异常，并返回相应的错误信息。

2.2 用户登录功能

2.2.1 功能概述

用户登录功能允许已注册用户通过输入用户名和密码进入系统。系统验证用户身份，生成JWT令牌，并根据用户角色分配相应的菜单权限，实现安全访问控制。

2.2.2 详细讲解功能实现流程

用户登录流程从前端Login.vue组件开始，用户输入用户名和密码后点击登录按钮，触发handleLogin方法。该方法验证表单后通过request.post("/user/login", loginForm)发送登录请求。

后端UserController接收请求，调用userService.login(user)处理登录逻辑。在UserService中，系统首先通过用户名查找用户，然后使用BCryptPasswordEncoder.matches()验证密码。验证成功后，调用menuService.getMenusByRoleCode获取用户权限菜单，并通过JwtTokenUtils.genToken生成JWT令牌。

登录成功后，前端通过userStore.setUserInfo(data)将用户信息和令牌保存到Pinia状态管理和localStorage中，实现持久化登录。然后根据用户角色决定跳转路径，管理员跳转到后台管理界面，普通用户跳转到前台首页。

2.2.3 关键代码讲解

1. **后端登录验证逻辑**
   【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public User login(User user) {
    User dbUser = getByUsername(user.getUsername());
    // 用户存在性检查已经在 getByUsername 中处理
    if
(dbUser.getStatus().equals(AccountStatus.PENDING_REVIEW.getValue())) {
        throw new ServiceException("账号正在审核");
    }
    if (!bCryptPasswordEncoder.matches(user.getPassword(),
dbUser.getPassword())) {
        throw new ServiceException("用户名或密码错误");
    }

    // 获取用户的菜单权限
    List<Menu> menus =
menuService.getMenusByRoleCode(dbUser.getRoleCode());
```

半夜撕代码版权所有

```
    String token =
JwtTokenUtils.genToken(String.valueOf(dbUser.getId()),
dbUser.getPassword());

    dbUser.setMenuList(menus);
    dbUser.setToken(token);
    return dbUser;
}
```

这段代码实现了用户登录验证和权限获取：首先验证用户状态和密码，然后获取用户的菜单权限，生成JWT令牌，最后将权限和令牌附加到用户对象返回。

2. 前端状态管理和路由跳转
【vue3/src/views/auth/Login.vue】

```
const handleLogin = () => {
  loginFormRef.value.validate(async valid => {
    if (valid) {
      loading.value = true
      try {
        const res = await request.post("/user/login", loginForm, {
          successMsg: "登录成功",
          showDefaultMsg: true,
          onSuccess: async (data) => {
            // 设置用户信息
            userStore.setUserInfo(data)

            // 根据角色决定跳转路径
            if (data.roleCode !== 'USER') {
              userStore.setMenus(data.menuList || [])
              setBackRoutes()
              // 管理员跳转
              router.push('/back/dashboard')
            } else {
              // 普通用户跳转
              router.push('/')
            }
          }
        })
      } finally {
        loading.value = false
      }
    }
  })
}
```

这段代码展示了前端登录流程：首先验证表单，然后发送登录请求，成功后保存用户信息，并根据角色跳转到相应页面。登录状态通过Pinia和localStorage实现持久化。

半夜撕代码版权所有

2.3 用户管理功能

2.3.1 功能概述

用户管理功能允许管理员查看、添加、编辑和删除用户信息。支持分页查询、条件筛选、批量操作等功能，提供完整的用户生命周期管理能力。

2.3.2 详细讲解功能实现流程

管理员通过UserController的getUsersByPage方法查询用户列表，支持按用户名、姓名和角色筛选。添加用户调用createUser方法，与普通注册流程类似但可指定角色。修改用户信息通过updateUser方法实现，删除用户则调用deleteUserById或deleteBatch方法。

所有操作都有相应的权限检查和数据验证，确保操作安全和数据一致性。例如，修改和删除用户前会检查用户是否存在，修改用户名时会检查是否与其他用户冲突。

2.3.3 关键代码讲解

1. 分页条件查询
【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public Page<User> getUsersByPage(String username, String name, String
roleCode, Integer currentPage, Integer size) {
    LambdaQueryWrapper<User> queryWrapper = new LambdaQueryWrapper<>();

    // 添加查询条件
    if (StringUtils.isNotBlank(username)) {
        queryWrapper.like(User::getUsername, username);
    }
    if (StringUtils.isNotBlank(name)) {
        queryWrapper.like(User::getName, name);
    }
    if (StringUtils.isNotBlank(roleCode)) {
        queryWrapper.eq(User::getRoleCode, roleCode);
    }

    return userMapper.selectPage(new Page<>(currentPage, size),
queryWrapper);
}
```

这段代码实现了用户的分页条件查询：根据前端传入的用户名、姓名、角色构建查询条件，然后使用MyBatis-Plus的分页查询功能获取结果。

半夜撕代码版权所有

2.4 密码管理功能

2.4.1 功能概述

密码管理功能包括密码修改和密码重置两个子功能。密码修改允许已登录用户更新自己的密码，密码重置允许忘记密码的用户通过邮箱验证重新设置密码。

2.4.2 详细讲解功能实现流程

用户修改密码时，需提供旧密码和新密码。系统首先验证旧密码是否正确，然后加密存储新密码。密码重置功能则通过邮箱确认用户身份，直接更新用户密码。

2.4.3 关键代码讲解

1. 密码修改功能
   【springboot/src/main/java/org/example/springboot/service/UserService.java】

```
public void updatePassword(Long id, UserPasswordUpdateDTO update) {
    User user = userMapper.selectById(id);
    if (user == null) {
        throw new ServiceException("用户不存在");
    }

    // 验证旧密码
    if (!bCryptPasswordEncoder.matches(update.getOldPassword(),
user.getPassword())) {
        throw new ServiceException("原密码错误");
    }

    // 更新新密码

 user.setPassword(bCryptPasswordEncoder.encode(update.getNewPassword())
);
    if (userMapper.updateById(user) <= 0) {
        throw new ServiceException("密码修改失败");
    }
}
```

这段代码实现了密码修改功能：首先验证用户存在性和旧密码正确性，然后对新密码进行加密并更新到数据库。

半夜撕代码版权所有

三、总结

3.1 用户注册功能实现

用户注册功能通过前端Register.vue组件收集用户信息，进行客户端表单验证后提交到后端UserController的createUser方法。后端首先验证用户名和邮箱的唯一性，然后使用BCryptPasswordEncoder对密码进行加密，最后将用户信息保存到数据库。整个流程确保了用户数据的有效性和安全性，为用户提供简洁高效的注册体验。

3.2 用户登录功能实现

用户登录功能通过前端Login.vue组件收集用户凭证，验证后提交到后端UserController的login方法。后端使用BCryptPasswordEncoder.matches()验证密码，并通过JwtTokenUtils.genToken()生成JWT令牌，同时获取用户菜单权限。登录成功后，前端将用户信息和令牌存储在Pinia状态管理和localStorage中，实现持久化登录，并根据用户角色导航至相应页面。

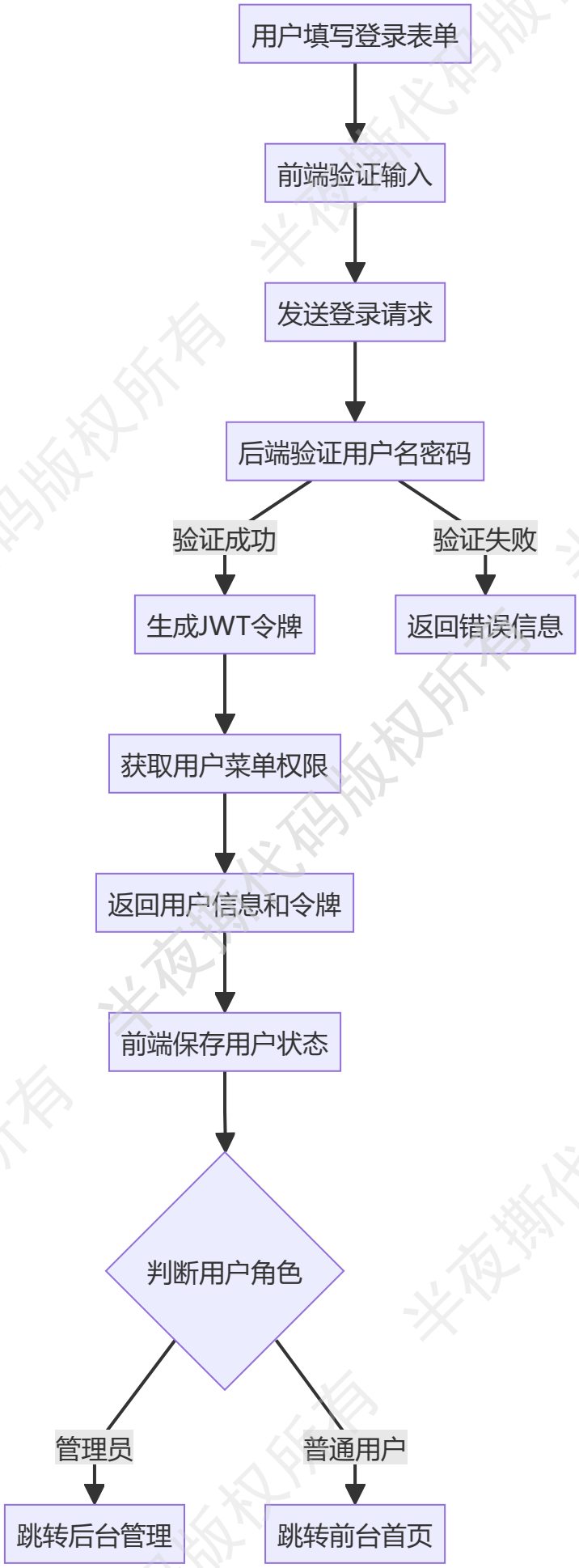
3.3 用户管理功能实现

用户管理功能提供了完整的用户CRUD操作，包括分页条件查询、添加、编辑和删除用户。管理员可以通过条件筛选快速定位用户，查看用户详情，修改用户信息，或删除用户账号。所有操作都有严格的权限控制和数据验证，确保系统安全和数据一致性。

3.4 密码管理功能实现

密码管理功能支持用户修改密码和忘记密码重置两种场景。修改密码时，系统先验证旧密码正确性，然后更新新密码；忘记密码时，通过邮箱验证用户身份后直接更新密码。所有密码都使用BCrypt算法加密存储，保证了用户账号的安全性。

半夜撕代码版权所有


用户填写登录表单
前端验证输入
发送登录请求
后端验证用户名密码
验证成功
验证失败
生成JWT令牌
返回错误信息
获取用户菜单权限
返回用户信息和令牌
前端保存用户状态
判断用户角色
管理员
普通用户
跳转后台管理
跳转前台首页

半夜撕代码版权所有